

СОВРЕМЕННЫЕ ТЕХНОЛОГИИ РАЗРАБОТКИ WEB- ПРИЛОЖЕНИЙ

ЛЕКЦИЯ 6В

Текст, таблицы, анимация

Переходы

Анимация

Медиа-запросы

Разное

Переходы

По умолчанию при использовании псевдокласса **:hover** изменение свойств происходит мгновенно.

Переходы в CSS позволяют плавно перейти от одного состояния элемента к другому.
отдельные **свойства** анимируются от **начального** состояния до **конечного (только два состояния)**.

CSS-свойства вида **transition-*** позволяют настроить продолжительность и метод перехода.

- **transition-property**: какие **свойства** анимируются;
- **transition-duration**: **как долго** длится анимация;
- **transition-timing-function**: как вычисляются **промежуточные состояния**;
- **transition-delay**: анимация начинается **через** некоторое время.

Можно установить каждое свойство CSS по отдельности или использовать сокращённую версию: **transition**.

Тогда, только **transition-duration** **обязательно**.

Продолжительность перехода

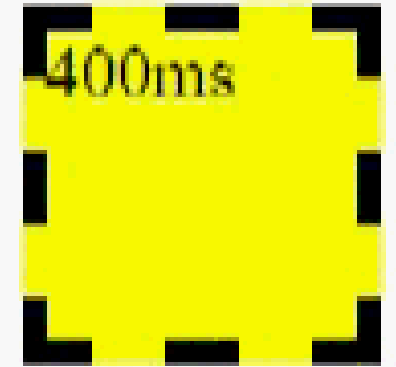
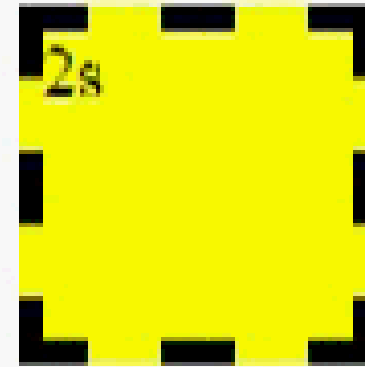
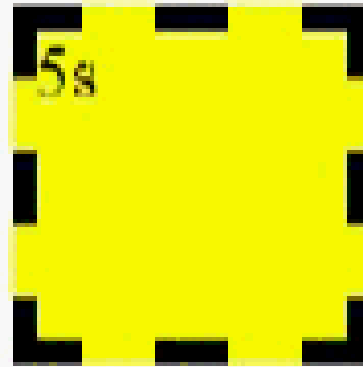
Продолжительность перехода задаётся при помощи свойства **transition-duration** (s/ms).

Можно указать время ожидания перед запуском перехода при помощи свойства **transition-delay**.

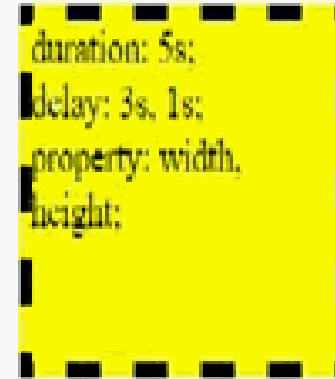
При описании перехода можно конкретизировать свойства, которые будут анимироваться, указав их имена через запятую в CSS-свойстве **transition-property**.

При таком подходе *возможно* указание нескольких значений для длительности и задержки (персонально для описанных анимированных свойств).

```
div { width : 70px;  
height : 70px;  
background-color : yellow;  
border : 5px dashed black;  
display : inline-block;  
/* (выстраиваем в линейку) */  
margin-right : 15px; }  
div:hover { height : 200px; }  
.test {transition-duration : 5s;}  
.test2 {transition-duration : 2s;}  
.test3 {transition-duration : 400ms;}
```



```
div:hover {width : 100%;  
height : 200px;  
background-color : rgba(255,0,0,.7);  
}
```



```
.test {transition-duration : 5s;  
/* длительность перехода */  
transition-delay : 3s, 1s;  
/* устанавливаем задержку перехода (1 значение = 1 значение  
свойства transition-property (width), 2 значение = 2  
значение свойства transition-property (height) */  
transition-property : width, height;  
/* устанавливаем свойства, которые  
получат переходный эффект */}
```


Переходы – функция изменения

Закон, по которому происходит изменение значений при переходе, определяется некой функцией.

Вид этой функции можно настроить при помощи **transition-timing-function**, используя при этом математическую функцию (кубическую кривую Безье).

Допускается более одного значений функций через запятую перед началом эффекта перехода

каждое новое значение функции соответствует значению в свойстве **transition-property**.

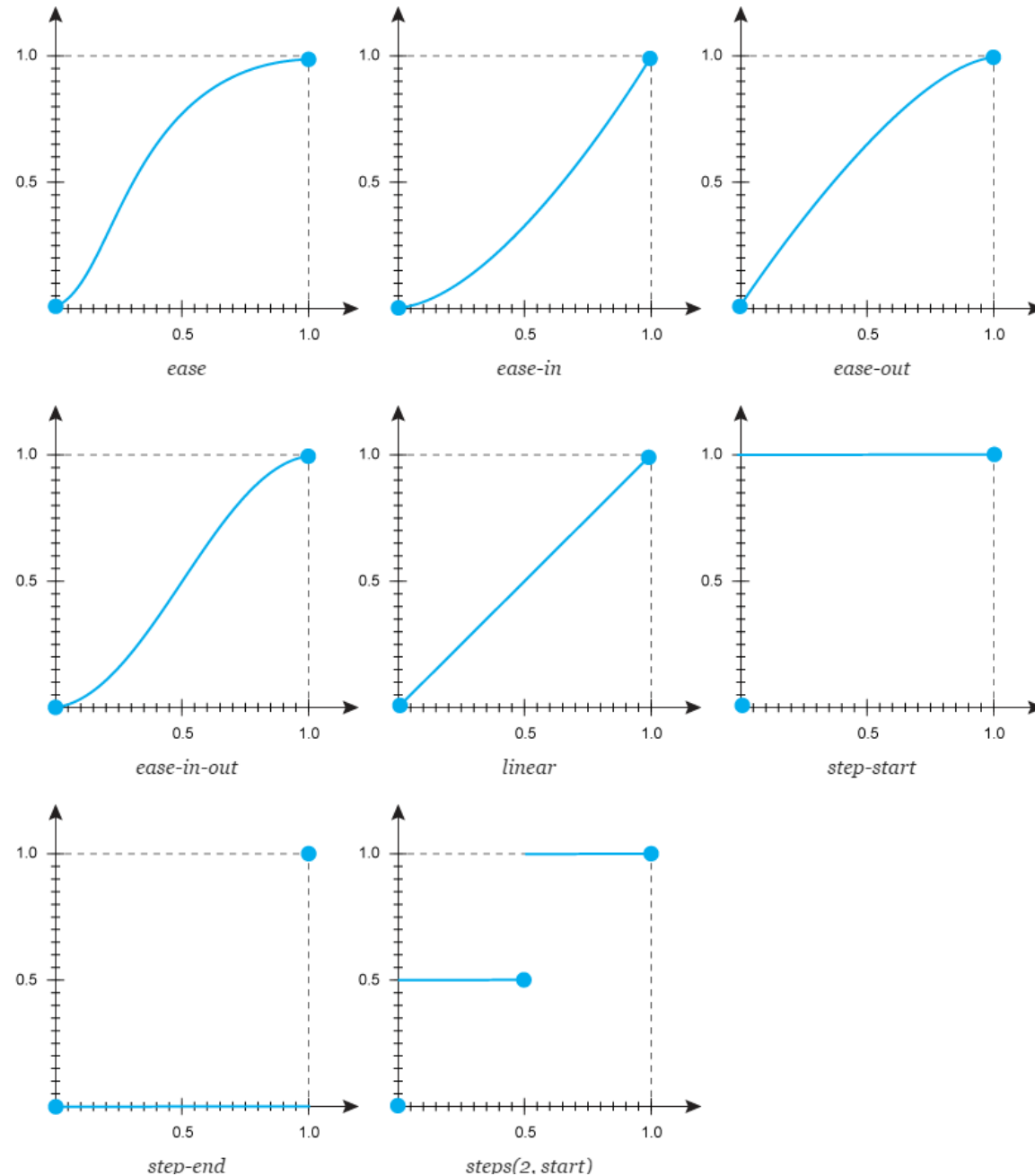
Если список функций меньше, чем количество значений (свойств), **transition-property**, то значения функций для оставшихся свойств будут использованы еще раз (по кругу).

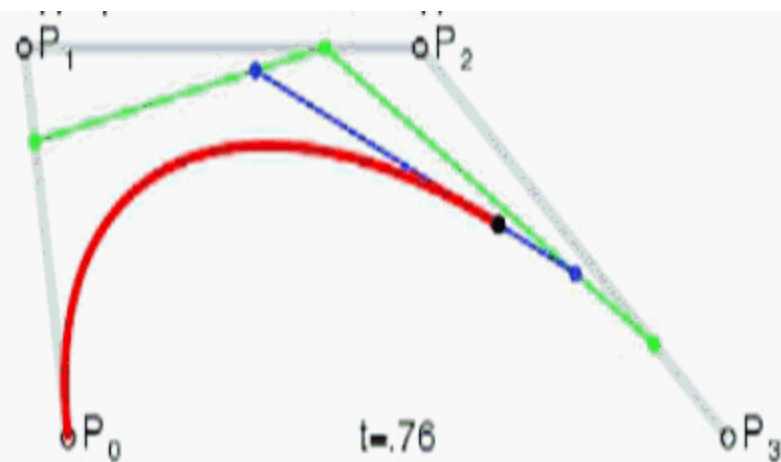
Если определено больше функций, то список функций усекается до количества значений этого свойства.

Для **transition-timing-function** существуют
преопределённые значения
(**ease** – по умолчанию):

cubic-bezier(0.25,0.1,0.25,1)

Эффект перехода начинается
медленно, затем
незначительно ускоряется и в
конце опять замедляется





А ещё функцию можно определить установив пользовательские значения кривой Безье:

Первые два значения — координаты x и y первой точки, а вторые два значения — координаты x и y второй точки

```
transition-timing-function: cubic-  
bezier(n,n,n,n);
```

(что такое *кривая Безье*:

http://www.wikiwand.com/ru/%D0%9A%D1%80%D0%B8%D0%B2%D0%B0%D1%8F_%D0%91%D0%B5%D0%B7%D1%8C%D0%B5

<http://habrahabr.ru/post/220715/>)

DONATE

About



Made by Lea Verou with care

PROGRESSION

TIME

cubic-bezier(.11, .81, 1, .99) SAVE

Preview & compare GO!

Duration: 0.8 seconds

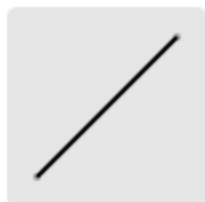


Library IMPORT EXPORT

Click on a curve to compare it with the current one.



ease



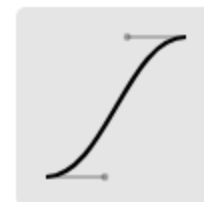
linear



ease-in



ease-out



ease-in-out

<http://cubic-bezier.com/#.11,.81,1,.99>

.test {transition-timing-function : ease; }

**/* эффект перехода начинается медленно,
затем незначительно ускоряется и в конце опять замедляется */**

.test2 {transition-timing-function : linear; }

/* эффект перехода с одинаковой скоростью от начала до конца. */

**.test3 {transition-timing-function : ease-in; } /* эффект перехода
с медленного старта */**

.test4 {transition-timing-function : ease-out; }

/* эффект перехода с медленным окончанием */

.test5 {transition-timing-function : ease-in-out; }

/* эффект перехода с медленного старта и медленным окончанием */

.test6 {transition-timing-function : cubic-bezier(0.5,0.05,0.7,0.5);; }

/* пользовательские значения в кубической функции Безье */

ease

linear

ease-in

ease-out

ease-in-out

cubic-bezier

```
div { left: 0; position: relative; transition: 1s; }
```

```
main:hover div { left: 200px; }
```

```
.ease { transition-timing-function: ease; } /* по умолчанию */
```

```
.linear { transition-timing-function: linear; } /* Постоянная скорость */
```

```
.ease-in { transition-timing-function: ease-in; }
```

```
.ease-out { transition-timing-function: ease-out; }
```

```
.ease-in-out { transition-timing-function: ease-in-out; }
```

ease: медленно начинается, быстро в середине, медленно завершается



linear: постоянная скорость



ease-in: медленно начинается, быстро завершается



ease-out: быстро начинается, медленно завершается



ease-in-out: похоже на ease, но с более выраженным ускорением/замедлением



Универсальное свойство `transition` позволяет одновременно задать `transition-property`, `transition-duration`, `transition-timing-function` и `transition-delay`.

```
transition: property duration timing-function delay;
```

```
transition: width 2s ease 100ms, height 1s linear 2s, background 0s 4s;
```

/ Для свойства width длительность эффекта перехода равна 2 секунды, при этом эффект перехода начинается медленно, затем незначительно ускоряется и в конце опять замедляется (ease) и это всё происходит с предварительной задержкой в 100 миллисекунд.*

Для свойства height длительность эффекта перехода равна 1 секунде, при этом эффект перехода происходит с одинаковой скоростью от начала до конца (linear) и это всё происходит с предварительной задержкой в 2 секунды.

*Свойство background будет применено с задержкой 4 секунды. */*



Анимация

В CSS для построения анимации используются последовательности ключевых кадров.

Ключевой кадр (КК) – это набор свойства и их значений, снабжённый **относительной** временной отметкой (%).

Анимация – плавный переход от одного ключевого кадра к другому с автоматическим вычислением значений промежуточных кадров.

Ключевые кадры

Один КК – это совокупность относительной временной отметки (в процентах) и блока объявления свойств.

Первый КК из последовательности имеет (обычно) отметку **0%**, а последний – отметку **100%**.

Вместо отметок **0%** и **100%** можно использовать слова **from** и **to** соответственно.

Ключевые кадры

Нюанс 1: Первый КК может иметь отметку больше 0% .
В этом случае анимация начнётся с задержкой.
Аналогичные соображения о последнем кадре!

Нюанс 2: Не все CSS-свойства поддаются анимации.
Список можно найти здесь:

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_animated_properties

Набор ключевых кадров

Для создания набора ключевых кадров используется правило `@keyframes`.

Указывается *имя набора* (уникальное) и кадры.

Правило `@keyframes` включает имя анимации, любое число контрольных точек, а также свойства, предназначенные для анимации.

```
@keyframes slide {  
  0% { left: 0; top: 0; }  
  50% { left: 244px; top: 100px; }  
  100% { left: 488px; top: 0; } }
```



Применение анимации

Чтобы применить анимацию к выбранным селектором элементам, используются свойства вида **animation-***:

animation-name	Назначает набор ключевых кадров для элемента
animation-duration	Общая продолжительность анимации.
animation-delay	Задержка до старта анимации.
animation-timing-function	Функция времени анимации.

HTML

```
<div class="stage"> <figure class="ball"></figure> </div>
```

CSS

```
@keyframes slide {  
0% { left: 0; top: 0; }  
50% { left: 244px; top: 100px; }  
100% { left: 488px; top: 0; } }  
.stage { height: 150px; position: relative; }  
.ball { height: 50px; position: absolute; width: 50px; }  
.stage:hover  
.ball { animation-name: slide;  
animation-duration: 2s;  
animation-timing-function: ease-in-out;  
animation-delay: .5s; }
```



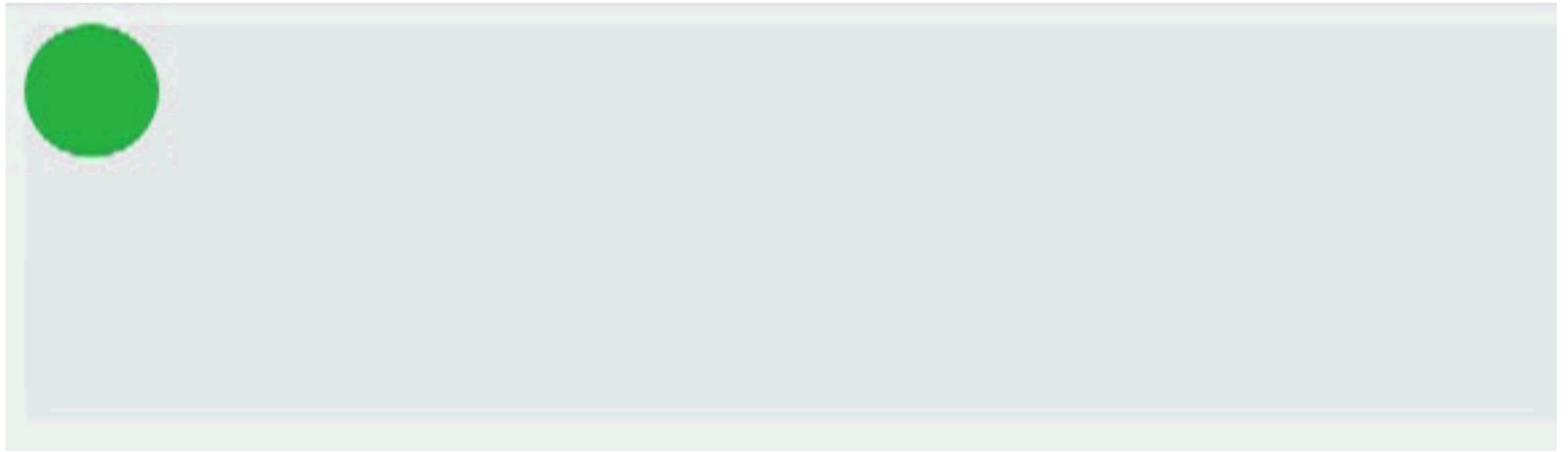
Настройка анимации

Свойство `animation-iteration-count` определяет, сколько раз проигрывать цикл анимации. Значением является положительное число или `infinite` – бесконечно будет проигрываться.

Свойство `animation-direction` задаёт направление выполнения анимации по набору ключевых кадров (как определено `@keyframes`):

<code>normal</code>	начало – конец
<code>alternate</code>	начало – конец, потом конец – начало
<code>reverse</code>	конец – начало
<code>alternate-reverse</code>	конец – начало, начало – конец

```
.stage:hover .ball {  
  animation-name: slide;  
  animation-duration: 2s;  
  animation-timing-function: ease-in-out;  
  animation-delay: .5s;  
  animation-iteration-count: infinite;  
  animation-direction: alternate; }
```



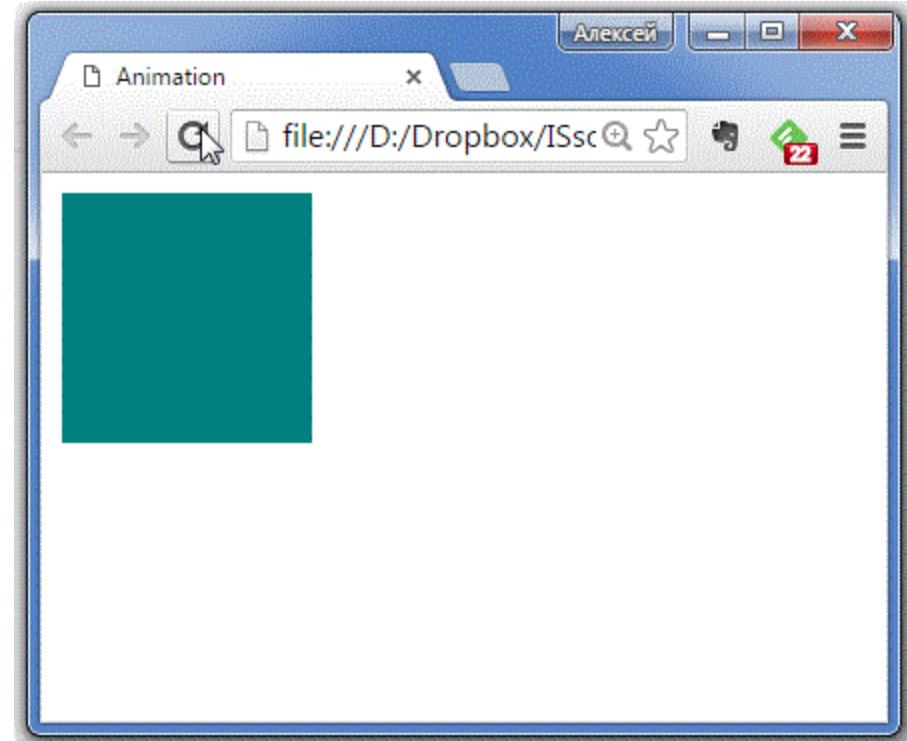
Настройка анимации

По умолчанию после анимации стиль элемента возвращается к исходному состоянию. Свойство **animation-fill-mode** позволяет изменить это.

none	К элементу не применяются какие-либо стили, связанные с анимацией.
forwards	По окончании анимации применяется стиль последнего ключевого кадра. Что это за кадр, зависит от значений свойств animation-direction и animation-iteration-count .
backwards	Стиль первого ключевого кадра применяется и остаётся на протяжении периода заданного animation-delay . Первый ключевой кадр определяется на основании значения animation-direction .
both	К элементу применяются оба правила: forwards и backwards .

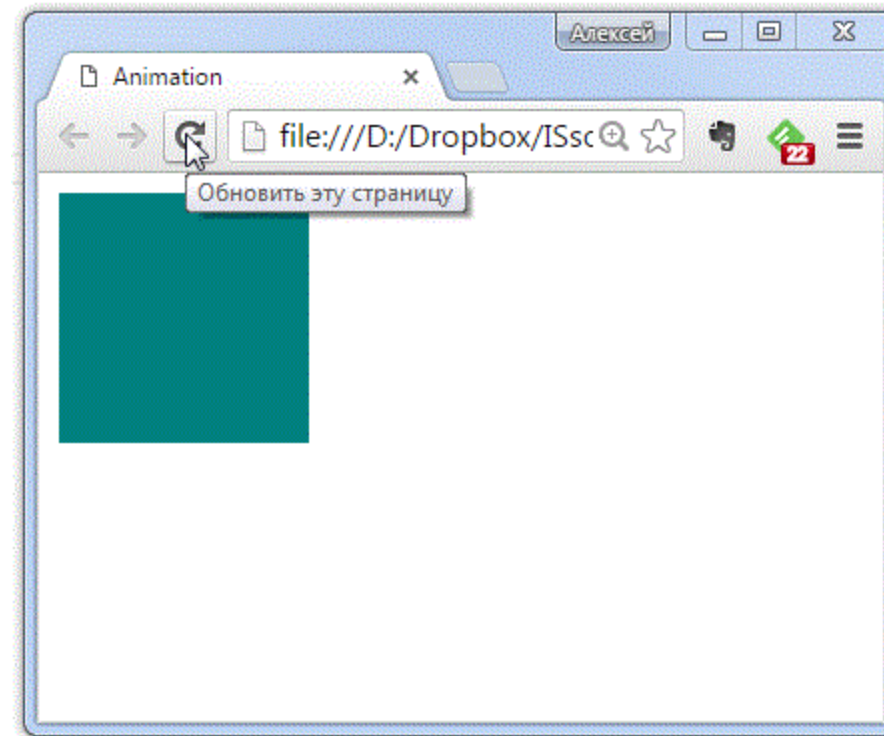
Настройка анимации

```
#box {  
  width: 100px;  
  height: 100px;  
  background-color: teal;  
  animation-name: grow;  
  animation-duration: 3s;  
  animation-delay: 1s;  
  animation-fill-mode: forwards;  
}
```



Настройка анимации

```
#box {  
  width: 100px;  
  height: 100px;  
  background-color: teal;  
  animation-name: grow;  
  animation-duration: 3s;  
  animation-delay: 1s;  
  animation-fill-mode: backwards;  
}
```



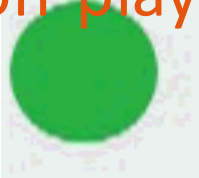
Настройка анимации

Свойство `animation-play-state` определяет, проигрывать анимацию (`running`) или поставить её на паузу (`paused`).

Часто используется с псевдоклассами (`:hover`) или через клиентские скрипты.

свойство **animation-play-state** устанавливается на паузу, когда stage активен при щелчке по нему. анимация временно приостанавливается, пока не отпустится кнопка мыши.

```
.stage:hover .ball {  
  animation-name: slide;  
  animation-duration: 2s;  
  animation-timing-function: ease-in-out;  
  animation-delay: .5s;  
  animation-iteration-count: infinite;  
  animation-direction: alternate; }  
.stage:active .ball { animation-play-state: paused; }
```



Настройка анимации

Настройку анимации можно выполнить при помощи универсального свойства **animation**.

```
animation: animation-name || animation-duration ||  
             animation-timing-function || animation-delay ||  
animation-iteration-count || animation-direction ||  
             animation-fill-mode || animation-play-state
```



```
.test { width: 75px;
height: 75px;
border-radius: 50px;
position: relative; /* элемент с относительным позиционированием
(position:static - по умолчанию, значение свойства left не повлияли бы на
позиционирование элемента) */
animation: iliketomoveit 5s infinite; /* имя анимации, продолжительность
анимации, будет повторяться бесконечно */ }
@keyframes iliketomoveit {
  0% {left: 0px;}
  25% {left: 400px;
background: red;}
  75% {left: 200px;}
  100% {left: 0px;
background: green;} }
```



Библиотеки для анимации

[Animate.css](#)

[Bounce.js](#)

[Magic Animations](#)

[DynCSS](#)

[CSShake](#)

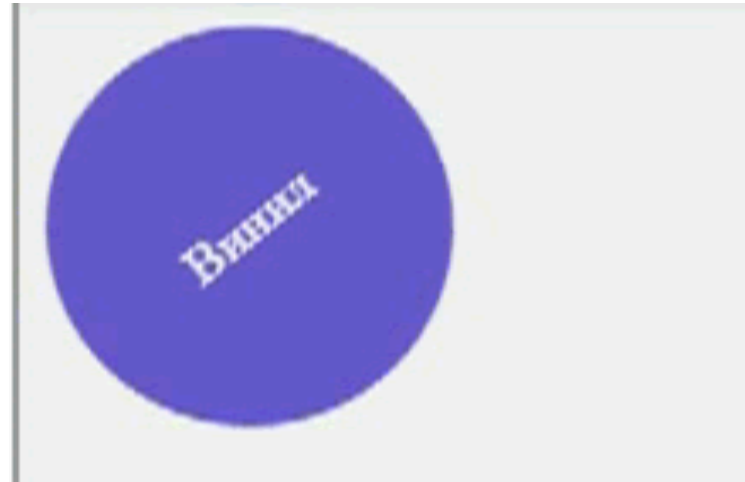
Анимации vs переходы

- ☐ Описать переход проще
- ☐ Переходы шире поддерживаются браузерами (пока)
- ☐ Переход всегда срабатывает как ответная реакция на изменение CSS-свойства
- ☐ Анимацию можно зациклить, переход – нет
- ☐ У перехода отсутствуют промежуточные ключевые кадры

```
@keyframes translating {  
0% { transform: translate(0, 0); }  
25% { transform: translate(240px, 0); }  
50% { transform: translate(240px, 140px); }  
75% { transform: translate(0, 140px); }  
100% { transform: translate(0, 0); }  
}  
p { animation: translating 4s linear infinite; }
```



```
@keyframes rotating {  
0% { transform: rotate(0deg); }  
100% { transform: rotate(360deg); }  
}  
p { animation: rotating 4s linear infinite; }
```



Медиа-запросы

Медиа-запросы (media queries) – средство CSS, позволяющее применять разные стили в зависимости от типа или технических характеристик устройства отображения.

Пример: один стиль применяется для принтера, другой – для браузера с шириной окна меньше 500 пикселей, третий – во всех остальных случаях.

Медиа-запросы — синтаксис

Медиа-запрос в CSS-файле — это правило `@media`, за которым следует одно или несколько **выражений**, описывающих параметры устройства. Затем в фигурных скобках записывается набор правил CSS.

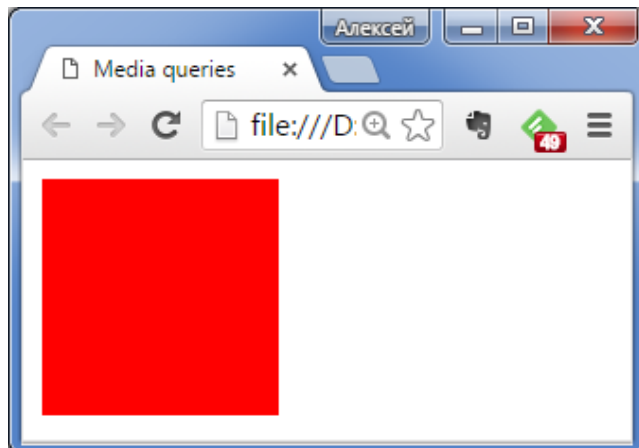
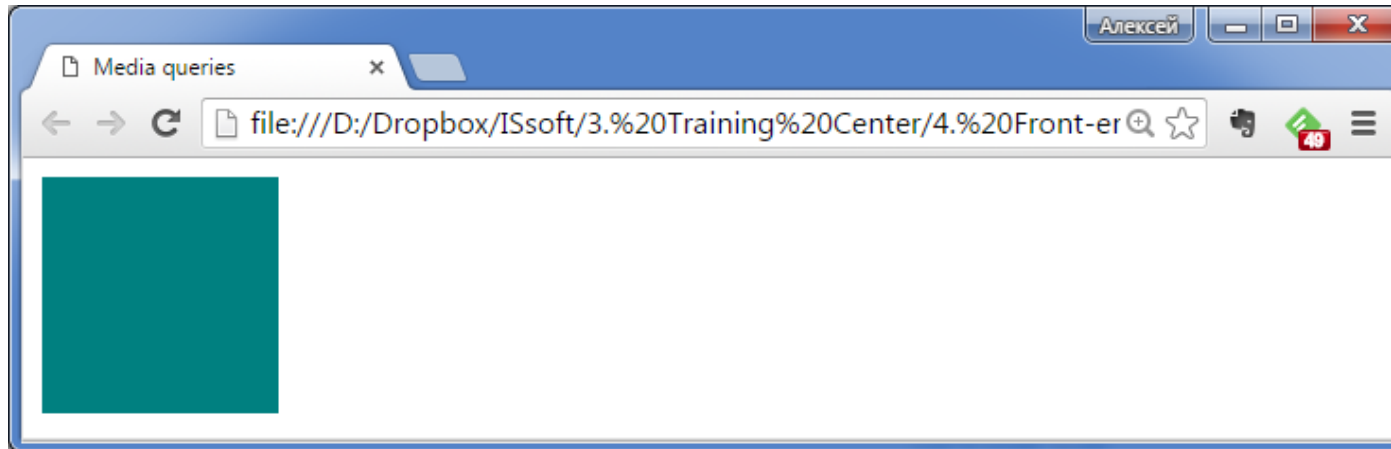
Правила применяются, если запрос **верен**, т.е. выражение (или **все выражения**) выполняются для данного устройства.

Медиа-запросы — пример

```
<style>
  #box {
    width: 100px;
    height: 100px;
    background-color: teal;
  }

  @media (max-width: 300px) {
    #box {
      background-color: red;
    }
  }
</style>
```


Медиа-запросы – пример



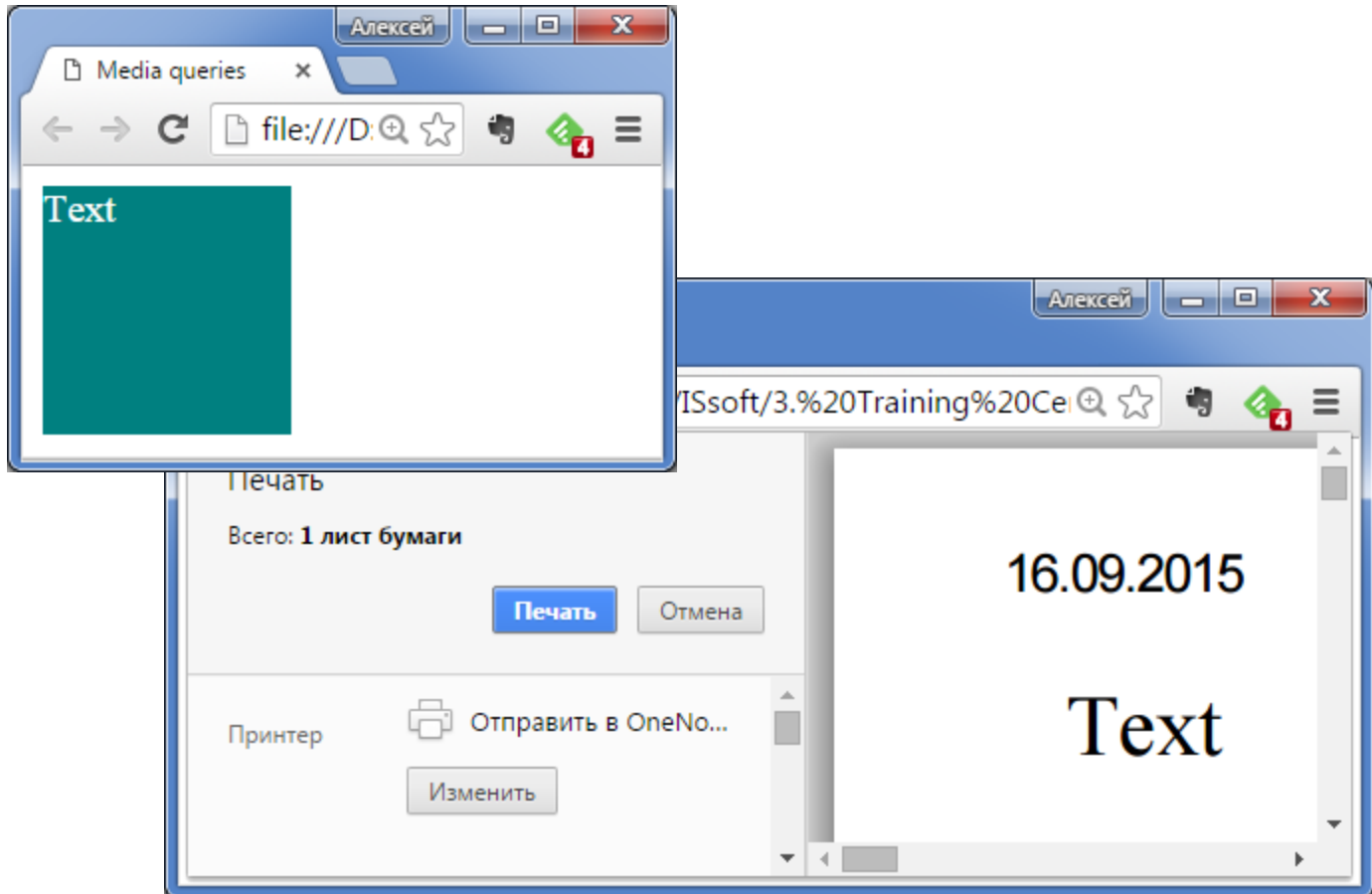
Устройства

Типы носителей и их описание

<code>all</code>	Все типы. Это значение используется по умолчанию.
<code>braille</code>	Устройства, основанные на системе Брайля.
<code>embossed</code>	Принтеры, использующие для печати систему Брайля.
<code>handheld</code>	Смартфоны и аналогичные им аппараты.
<code>print</code>	Принтеры и другие печатающие устройства.
<code>projection</code>	Проекторы.
<code>screen</code>	Экран монитора.
<code>speech</code>	Речевые синтезаторы, а также программы для воспроизведения текста вслух. Сюда, например, можно отнести речевые браузеры.
<code>tty</code>	Устройства с фиксированным размером символов (пример: телетайпы).
<code>tv</code>	Телевизоры.

Устройства – пример

```
#box {  
  width: 100px;  
  height: 100px;  
  background-color: teal;  
}  
  
@media screen {  
  #box {  
    color: white;  
  }  
}  
  
@media print {  
  #box {  
    color: black;  
  }  
}
```



Медиа-функции

Медиа-функции описывают технические характеристики устройства отображения (пример: разрешение экрана).

Многие функции содержат приставку **min-** и **max-**:

(max-width:400px) верно, если ширина окна браузера меньше, чем 400 пикселей

(min-width:600px) верно, если ширина окна больше, чем 600 пикселей

Некоторые медиа-функции

width min-width max-width	Ширина отображаемой области.
height min-height max-height	Высота отображаемой области.
aspect-ratio min-aspect-ratio max-aspect-ratio	Соотношение ширины и высоты отображаемой области. Значение указывается как два целых числа, разделённых слэшем (/).
color min-color max-color	Число бит на канал цвета. Если значение не указано, тогда проверяется что устройство цветное.
device-aspect-ratio device-height device-width	Ширина, высота и соотношение ширины и высоты для устройства (а не для отображаемой области). Есть варианты с min и max (min-device-height).
orientation	Альбомный (landscape) или портретный (portrait) режим.

Медиа-функции — примеры

```
/* цветное устройство */
```

```
@media (color) { . . . }
```

```
/* устройство в альбомном режиме */
```

```
@media (orientation: landscape) { . . . }
```

```
/* высота окна (браузера) меньше, чем 600px */
```

```
@media (max-height: 600px) { . . . }
```

Логические операторы

При описании медиа-запроса вызовы медиа-функций и типы носителей можно комбинировать, используя логические операторы:

<code>and</code>	«И»
<code>,</code> (запятая)	«ИЛИ», перечисляются запросы
<code>not</code>	«НЕТ»

При применении операторов можно указывать скобки.

Логические операторы — пример

/ устройство - экран и он цветной */*

@media screen and (color) { ... }

/ либо это экран, либо любое цветное устройство*/*

@media screen, (color) { ... }

/ внимание - это not (screen and (color)) */*

@media not screen and (color) { ... }

/ внимание - это (not screen) или color */*

@media not screen, (color) { ... }

Медиа-запросы и вёрстка

Полный пример здесь: <http://habrahabr.ru/post/119127/>

```
/* Размер экрана меньше 980px (резиновый макет) */  
@media screen and (max-width: 980px) { . . . }  
  
/* Размер экрана меньше 650px (одна колонка) */  
@media screen and (max-width: 650px) { . . . }  
  
/* Размер экрана меньше 480px (экран iPhone) */  
@media screen and (max-width: 480px) { . . . }
```

Медиа-запросы и элемент link

Все примеры до этого – работа в CSS (файл или **style**).

Но медиа-запрос можно указать в атрибуте **media** у элемента **link** (разные CSS-файлы):

```
<link rel="stylesheet" href="small.css"  
      media="screen and (max-width: 600px)" />
```

А ещё атрибут **media** есть у элементов **style**, **source** (аудио, видео, **picture**), элементов **a** (с **href**) и **area**.

Разное

В этой части рассмотрим несколько свойств и функций, не затронутых ранее.

Курсор

Свойство **cursor** устанавливает форму курсора, когда он находится в пределах элемента (вид курсора зависит от операционной системы).

	default		pointer		n-resize		s-resize
	crosshair		progress		ne-resize		sw-resize
	help		text		e-resize		w-resize
	move		wait		se-resize		nw-resize

Прозрачность

Свойство **opacity** определяет уровень прозрачности элемента. В качестве значения выступает число из диапазона $[0.0; 1.0]$.

Значение 0 соответствует полной прозрачности элемента, а 1 (по умолчанию) – его непрозрачности. Дробные числа устанавливают полупрозрачность.

Функция calc()

Используется для вычисления значения свойства и позволяет комбинировать разные единицы измерений. Допустимые операторы: + , - , * , / . Если значение не может быть вычислено, оно игнорируется.

```
.banner {  
    position: absolute;  
    left: 40px;  
    width: calc(100% - 80px);  
}
```

Функция attr()

Применяется для добавления значения атрибута HTML-элемента в стилевое свойство (сейчас – только **content**).

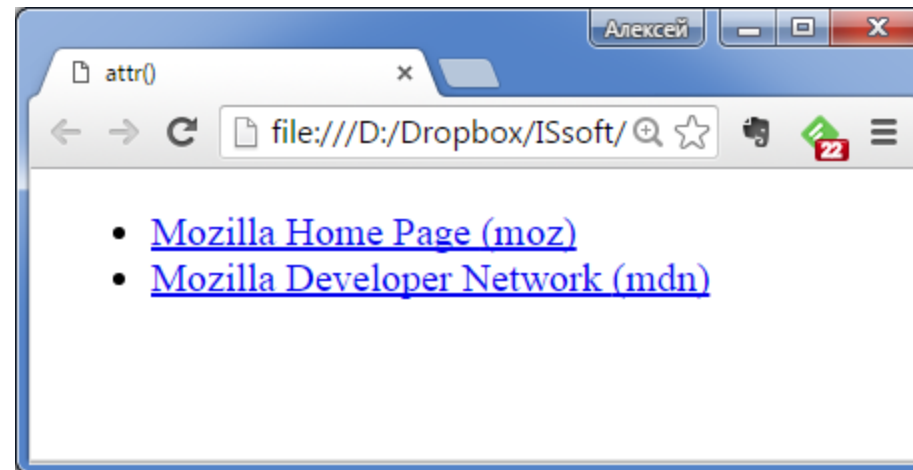
Синтаксис: **attr(<атрибут>, <тип>, <значение>)**

<атрибут>	Имя атрибута элемента.
<тип>	Тип значения свойства CSS. Допустимы следующие ключевые слова: string (строки), color (цвет), url (веб-адрес), integer (целое число), number (вещественное число), angle (угол), time (время), deg (градус), а также единицы CSS вроде em, px и др.
<значение>	Значение, которое будет использоваться в том случае, если атрибута нет или браузер не может его получить.

Функция attr() – пример

```
<style>
  a::after {
    content: " (" attr(id) ")";
  }
</style>
```

```
<ul>
  <li><a id="moz" href="http://www.mozilla.org/">
    Mozilla Home Page</a></li>
  <li><a id="mdn" href="https://developer.mozilla.org/">
    Mozilla Developer Network</a></li>
</ul>
```



CSS свойство filter

CSS свойство **filter** устанавливает визуальный эффект (фильтр) для элемента (например, размытие или изменение цвета).

Синтаксис:

```
filter:"none | blur() | brightness() | contrast() | drop-  
shadow() | grayscale() | hue-rotate() | invert() | opacity() |  
saturate() | sepia() | url()";
```

```
filter:"blur(10px) brightness(150%);
```

Шаблоны веб-сайтов W3.CSS

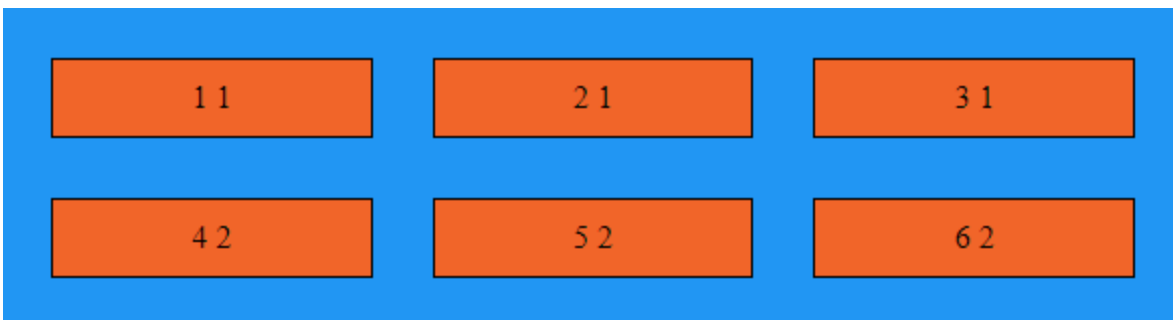
https://www.w3schools.com/css/css_rwd_templates.asp

CSS Grid Layout

Предлагает систему макета на основе сетки со строками и столбцами, что упрощает разработку веб-страниц без использования плавающих элементов и позиционирования.

Макет сетки состоит из родительского элемента с одним или несколькими дочерними элементами.

```
<div class='grid-container'>
  <div>1 1</div>
    <div>2 1</div>
    <div>3 1</div>
    <div>4 2</div>
    <div>5 2</div>
    <div>6 2</div>
</div>
```



```
.grid-container {
  background: rgb(0, 150, 208);
  display: grid;
  grid-template-columns: auto auto auto;
  background-color: #2196F3;
  padding: 10px;
}
```

```
.grid-container > div {
  background: rgb(241, 101, 41);
  border: 1px solid;
  margin: 15px;
  padding: 10px;
  text-align: center;
}
```

HTML-элемент становится контейнером сетки, если для его **display** свойства установлено значение **grid** или **inline-grid**

Все прямые дочерние *элементы* контейнера сетки автоматически становятся *элементами сетки* .

First name:		Department: Finance Human Resources Marketing Payroll Shipping	
Last name:			
Address:			
	Back	Cancel	Next

Столбцы, строки, линии и разрывы сетки

Column	Column	Column

Row			
Row			
Row			

	Column Gap	Column Gap	
Row Gap			
Row Gap			

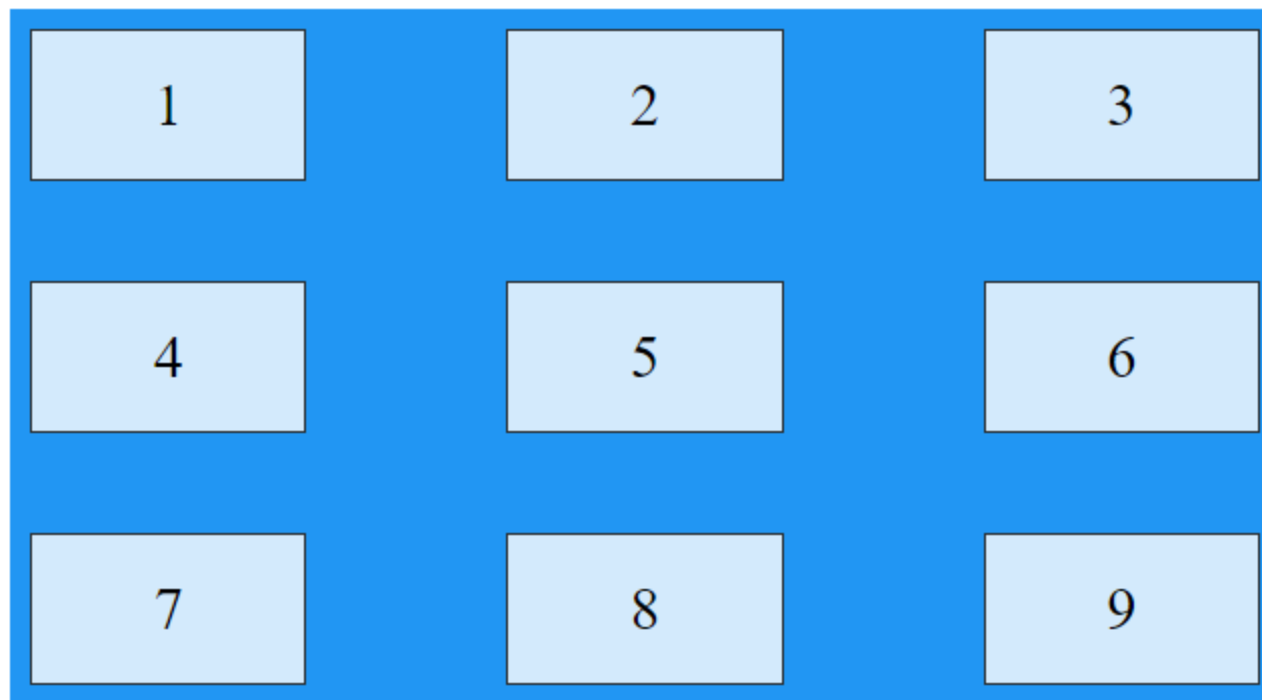
	Line 1	Line 2	Line 3	Line 4
Line 1				
Line 2				
Line 3				
Line 4				

grid-column-gap - устанавливает зазор между столбцами

grid-row-gap - устанавливает зазор между рядами

grid-gap - является сокращенным свойством для **grid-row-gap** и **grid-column-gap**

```
.grid-container {  
  display: grid;  
  grid-gap: 50px 100px;  
  grid-template-columns:  
  auto auto auto;  
  background-color:  
  #2196F3;  
  padding: 10px;  
}
```



Размещение элемента в контейнере сетки

```
.item1 {  
  grid-column-start: 1;  
  grid-column-end: 3;  
}
```

1		2
3	4	5
6	7	8

```
.item1 {  
  grid-row-start: 1;  
  grid-row-end: 3;  
}
```

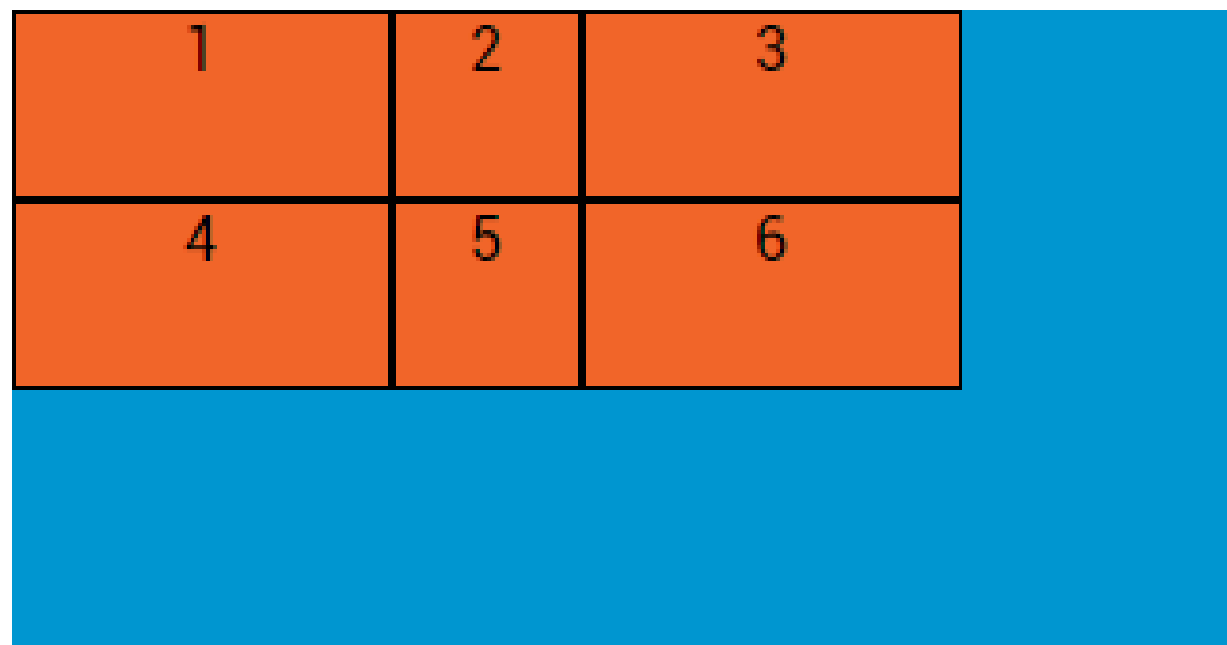
1	2	3
	4	5
6	7	8

Настройки контейнера

height [?]	300px
display [?]	grid
grid-template-columns [?]	100px 50px 100px
grid-template-rows [?]	50px 50px
grid-column-gap [?]	0
grid-row-gap [?]	0
grid-gap [?]	0
align-items [?]	stretch
justify-items [?]	stretch
place-items [?]	stretch
align-content [?]	stretch
justify-content [?]	flex-start
place-content [?]	stretch
grid-auto-columns [?]	auto
grid-auto-rows [?]	auto
grid-auto-flow [?]	row

Настройки элементов

count [?]	6
width [?]	auto
height [?]	auto
margin [?]	0px
padding [?]	0px



https://basicweb.ru/css/grid_generator.php

grid-template-rows (число (и высота) строк в макете сетки.

Значения представляют собой список, разделенный пробелами, где каждое значение определяет высоту соответствующей строки).

grid-template-columns (число (и ширина) столбцов в макете сетки.

Значения представляют собой список, разделенный пробелами, где каждое значение определяет размер соответствующего столбца.).

grid-template-areas (определяет шаблон макета сетки, ссылаясь при этом на имена областей элементов, которые задаются с помощью свойства **grid-area**).

grid-auto-rows (задает размер неявно созданных строк в контейнере сетки макета)

grid-auto-columns (задает размер неявно созданных столбцов в контейнере сетки макета)

grid-auto-flow (определяет, как работает алгоритм автоматического размещения для неявно созданных элементов макета, указывая как они будут размещены в сетке макета)

`grid: 40px 100px 120px / 1fr 2fr 100px;`

40px

100px

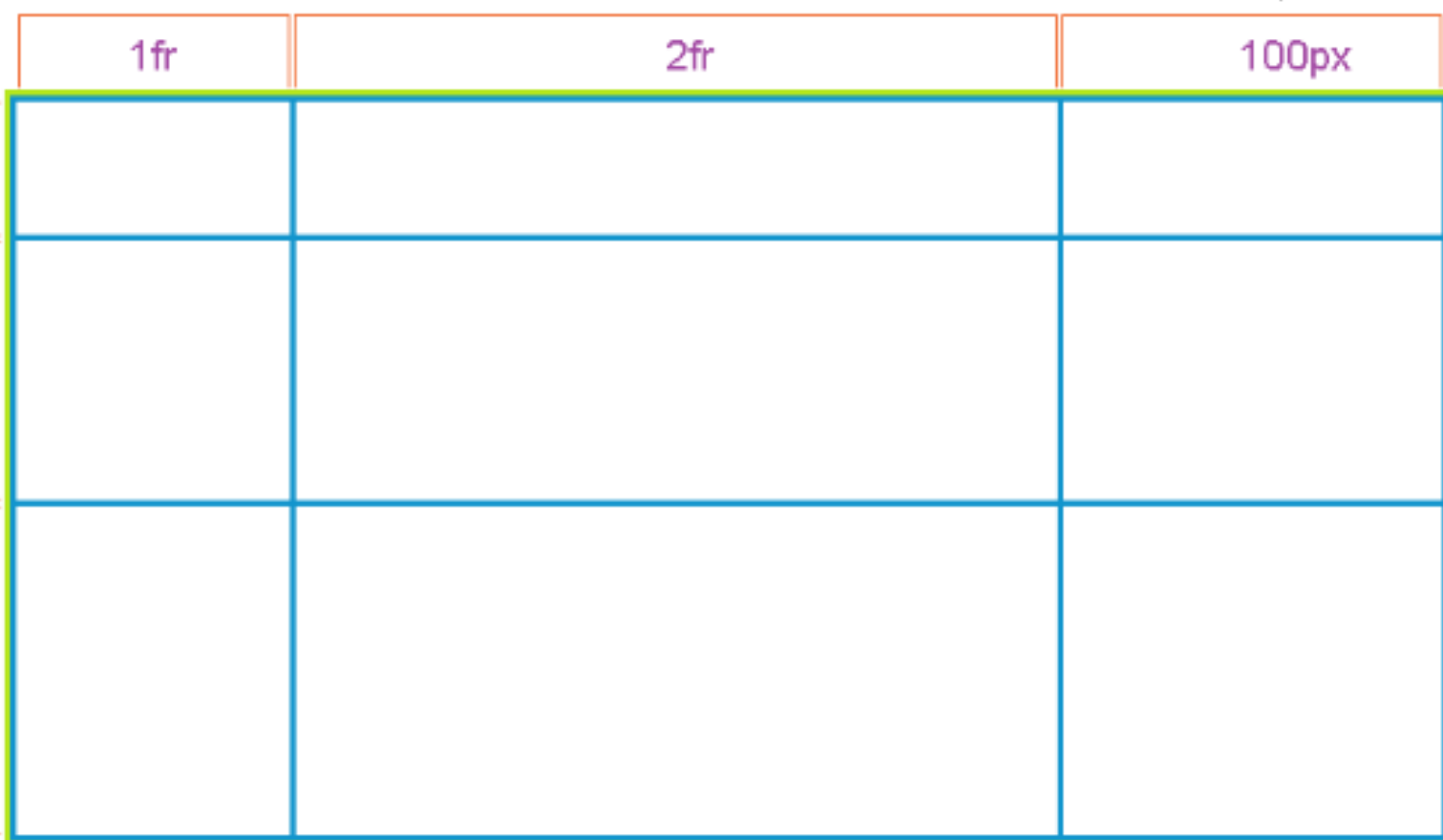
120px

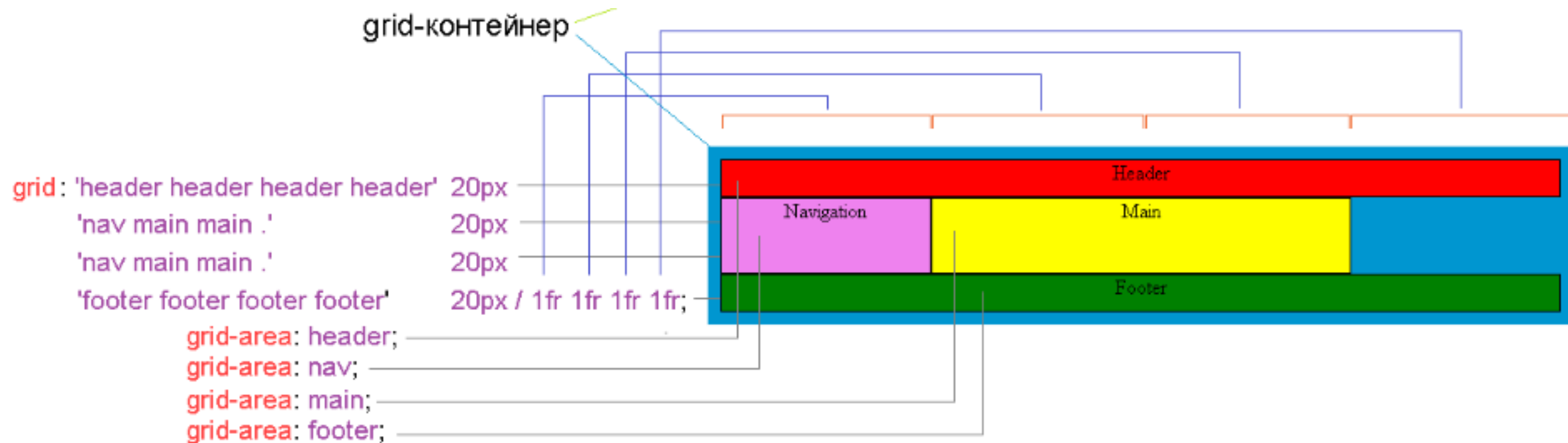
1fr

2fr

100px

grid-контейнер

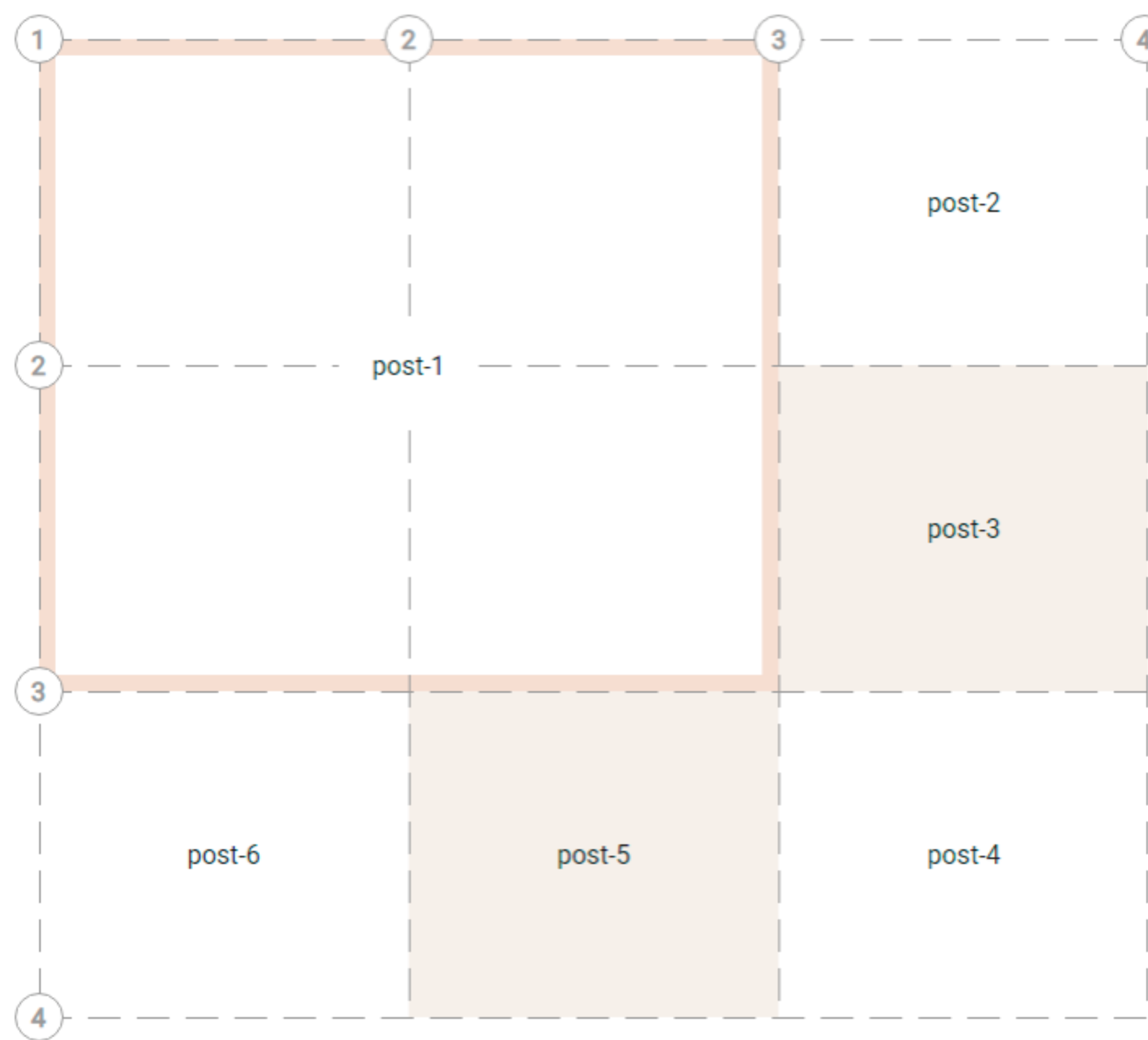




```
grid: 'header header header header' 20px /* определяем шаблон макета сетки,  
      ссылаясь при этом на имена областей элементов, кавычки обязательны */  
      'nav main main .' 20px /* точка обозначает элемент без имени */  
      'nav main main .' 20px /* точка обозначает элемент без имени */  
      'footer footer footer footer' 20px / 1fr 1fr 1fr 1fr; }
```

```
<div class = "grid-container">
  <div class = "item-a">Header</div>
  <div class = "item-b">Main</div>
  <div class = "item-c">Footer</div>
  <div class = "item-d">Navigation</div>
</div>
```

```
.grid-container > div {
background: rgb(241,101,41); /* цвет заднего фона */
border: 1px solid #000; /* сплошная граница размером 1 пиксель черного цвета */
text-align: center; /* горизонтальное выравнивание текста */ }
.item-a { grid-area: header; /* задаем имя элемента */
background: red !important; /* цвет заднего фона */ }
.item-b {
grid-area: main; /* задаем имя элемента */
background: yellow !important; /* цвет заднего фона */ }
.item-c { grid-area: footer; /* задаем имя элемента */
background: green !important; /* цвет заднего фона */ }
.item-d { grid-area: nav; /* задаем имя элемента */
background: violet !important; /* цвет заднего фона */ }
```



```
.grid-container {  
  display: grid;  
  grid-template-rows: 200px 200px 200px;  
  grid-template-columns: 1fr 1fr 1fr; }  
  
.post-1 {  
  grid-row-start: 1;  
  grid-row-end: 3;  
  grid-column-start: 1;  
  grid-column-end: 3; }  
  
.post-2 {  
  grid-row-start: 1;  
  grid-column-start: 3; }  
  
.post-3 {  
  grid-row-start: 2;  
  grid-column-start: 3; }  
  
.post-4 {  
  grid-row-start: 3;  
  grid-column-start: 3; }  
  
.post-5 {  
  grid-row-start: 3;  
  grid-column-start: 2; }  
  
.post-6 {  
  grid-row-start: 3;  
  grid-column-start: 1; }
```

CSS3 columns

многоколоночный макет, который позволяет организовать содержимое так, чтобы оно занимало несколько вертикальных контейнеров, подобно газете или журналу. Колонки могут содержать заголовки, текст, таблицы, картинки и любые другие inline-элементы

Количество и ширина колонок

Ширина колонок: свойство **column-width**

Количество колонок: свойство **column-count**

Установка колонок с помощью одного свойства **columns**

15 Most Charming Small Towns In England

1. *Berwick-upon-Tweed*



Berwick was founded as an Anglo-Saxon settlement during the time of the Kingdom of Northumbria, which was annexed by England in the 10th century. The area was for more than 400 years central to historic border wars between the Kingdoms of England and Scotland, and several times possession of Berwick changed hands between the two kingdoms. The last time it changed hands was when Richard of

Gloucester retook it for England in 1482. To this day many Berwickers feel a close affinity to Scotland.

Nowadays Berwick-upon-Tweed is much-visited for its highly visible history: medieval town walls, Elizabethan ramparts, 13th century castle ruins, its 17th century 'Old Bridge', town hall, Britain's earliest army barracks, England's northernmost hotel, amongst others.

2. *Rye*



Ancient Rye is all cobbled streets and tumbledown rows of houses by the sea. Originally part of the Cinque Ports Confederation, five strategic towns important for trade and military purposes in medieval times, today Rye is practically a living museum. Rye Castle, popularly known as Ypres Tower, was built in 1249 by Henry III to protect against frequent raids by the French; even older, the Norman-era St. Mary's Church looks over the town. Rye is also just a few minutes away from one of England's most famous beaches, Camber Sands, a two-mile-long playground for kitesurfers and beachlovers.


```
.container {
```

```
-webkit-columns: 150px 3; /* для поддержки ранних версий браузеров */
```

```
-moz-columns: 150px 3; /* для поддержки ранних версий браузеров */
```

```
columns: 150px 3; /* указываем оптимальную ширину и количество колонок на которое необходимо разбить элемент */ }
```

```
<div class = "container">
```

Этот гигантский грызун представляет собой жирного зверька с продолговатым телом, покрытым жёсткой лохматой шерстью пёстрой коричневой расцветки. Передние лапы у капибары длиннее задних, массивный огузок не имеет хвоста, и поэтому у неё всегда такой вид, будто она вот-вот собирается сесть. У неё крупные лапы с широкими перепончатыми пальцами, а когти на передних лапах, короткие и тупые, удивительно напоминают миниатюрные копыта. Вид у неё весьма аристократический: её плоская широкая голова и тупая, почти квадратная морда имеют благодушно-покровительственное выражение, придающее ей сходство с задумчивым львом. По земле капибара передвигается характерной шаркающей походкой или скачет вразвалку галопом, в воде же плавает и ныряет с поразительной лёгкостью и проворством. Капибара — флегматичный добродушный вегетарианец, лишённый ярких индивидуальных черт, присущих некоторым его сородичам, но этот недостаток восполняется у неё спокойным и дружелюбным нравом.

```
</div>
```

Этот гигантский грызун представляет собой жирного зверька с продолговатым телом, покрытым жёсткой лохматой шерстью пёстрой коричневой расцветки. Передние лапы у капибары длиннее задних, массивный огузок не имеет хвоста, и поэтому у неё всегда такой вид, будто она вот-вот собирается сесть. У неё крупные лапы с широкими перепончатыми пальцами, а когти на передних лапах, короткие

и тупые, удивительно напоминают миниатюрные копыта. Вид у неё весьма аристократический: её плоская широкая голова и тупая, почти квадратная морда имеют благодушно-покровительственное выражение, придающее ей сходство с задумчивым львом. По земле капибара передвигается характерной шаркающей походкой или скачет вразвалку галопом, в воде же плавает и ныряет с

поразительной лёгкостью и проворством. Капибара — флегматичный добродушный вегетарианец, лишённый ярких индивидуальных черт, присущих некоторым его сородичам, но этот недостаток восполняется у неё спокойным и дружелюбным нравом.

column-gap	разрыв между колонками. Если для колонок установлена разделительная линия с помощью свойства column-rule, то эта линия будет расположена посередине промежутка, а ее ширина не изменит общую ширину.
column-rule-color	цвет разделительной линии
column-rule-style	стиль разделительной линии
column-rule-width	ширина разделительной линии. Отрицательные значения не допускаются. Не работает без свойства column-rule-style
column-rule	сокращенная запись свойств column-rule-width column-rule-style column-rule-color
break-before, break-after и break-inside	разрывы колонок
column-span	охватывать несколько столбцов (none, all - все колонки). Указывается не для блока-контейнера, а для конкретного элемента внутри, например, для заголовка.
column-fill	контролирует заполнение колонок содержимым. Существует две стратегии заполнения колонок: колонки могут быть выровнены по высоте (balance, balance-all)
overflow	содержимое и разделительные линии, которые выходят за рамки колонок по краям многоколоночного контейнера

Flexbox — состоит из гибкого контейнера (flex container) и гибких элементов (flex items)

- вертикальное выравнивание блока внутри родительского элемента.
- размещение колонок макета одинаковой высоты независимо от наполнения их содержимым.
- расположение дочерних элементов контейнера в необходимом направлении, распределяя доступную ширину (высоту), не вызывая при этом переполнения родительского элемента.
- изменять порядок элементов независимо от порядка в *DOM*.

Поддержка браузерами

IE: 11.0, 10.0 -ms-

Firefox: 28.0, 18.0 -moz-

Chrome: 29.0, 21.0 -webkit-

Safari: 6.1 -webkit-

Opera: 12.1 -webkit-

iOS Safari: 7.0 -webkit-

Opera Mini: 8

Android Browser: 4.4, 4.1 -webkit-

Chrome for Android: 44

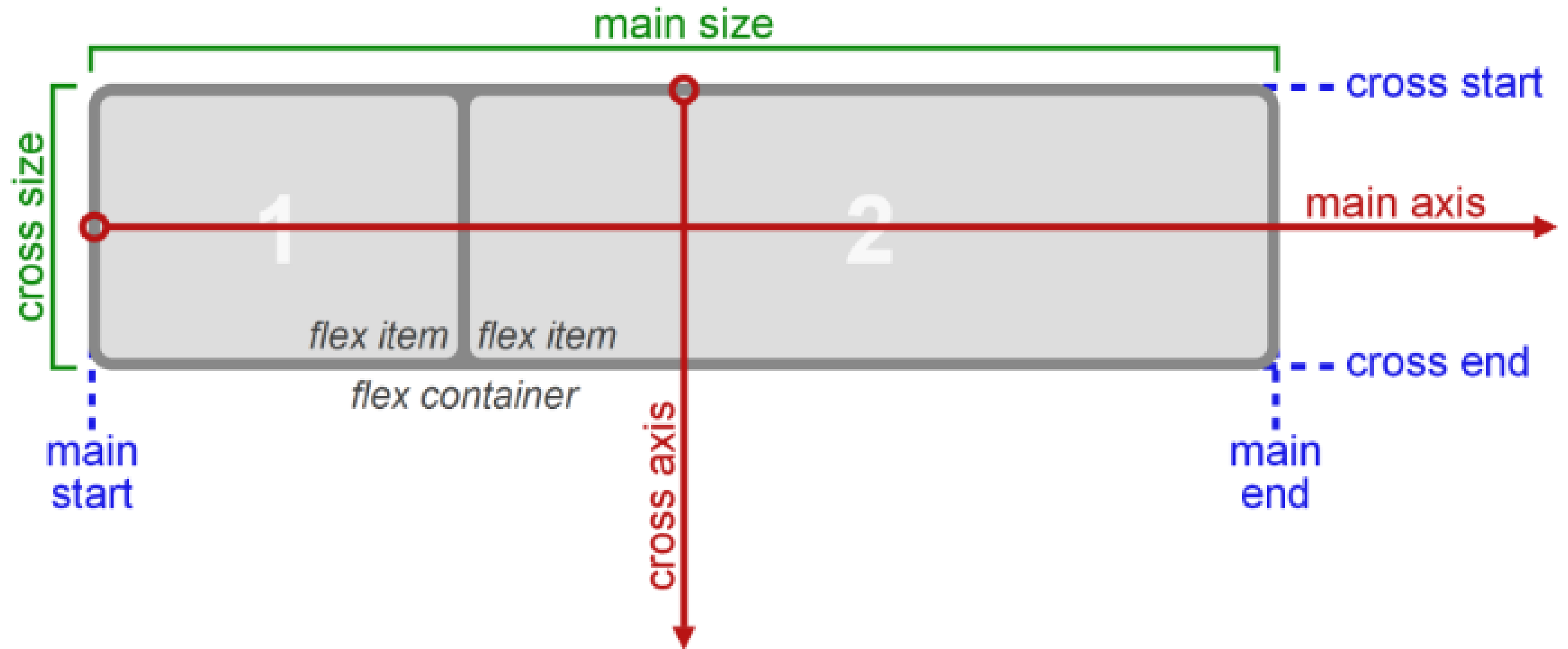
Флекс-контейнер – любой семантически осмысленный элемент.

```
section {  
    display: flex; // элемент отображается как блочный флекс контейнер  
}  
.flex-container {  
    /*генерирует flex-контейнер уровня блока*/  
    display: -webkit-flex;  
    display: flex;  
}  
.flex-container {  
    /*генерирует flex-контейнер уровня строки*/  
    display: -webkit-inline-flex;  
    display: inline-flex;  
}
```

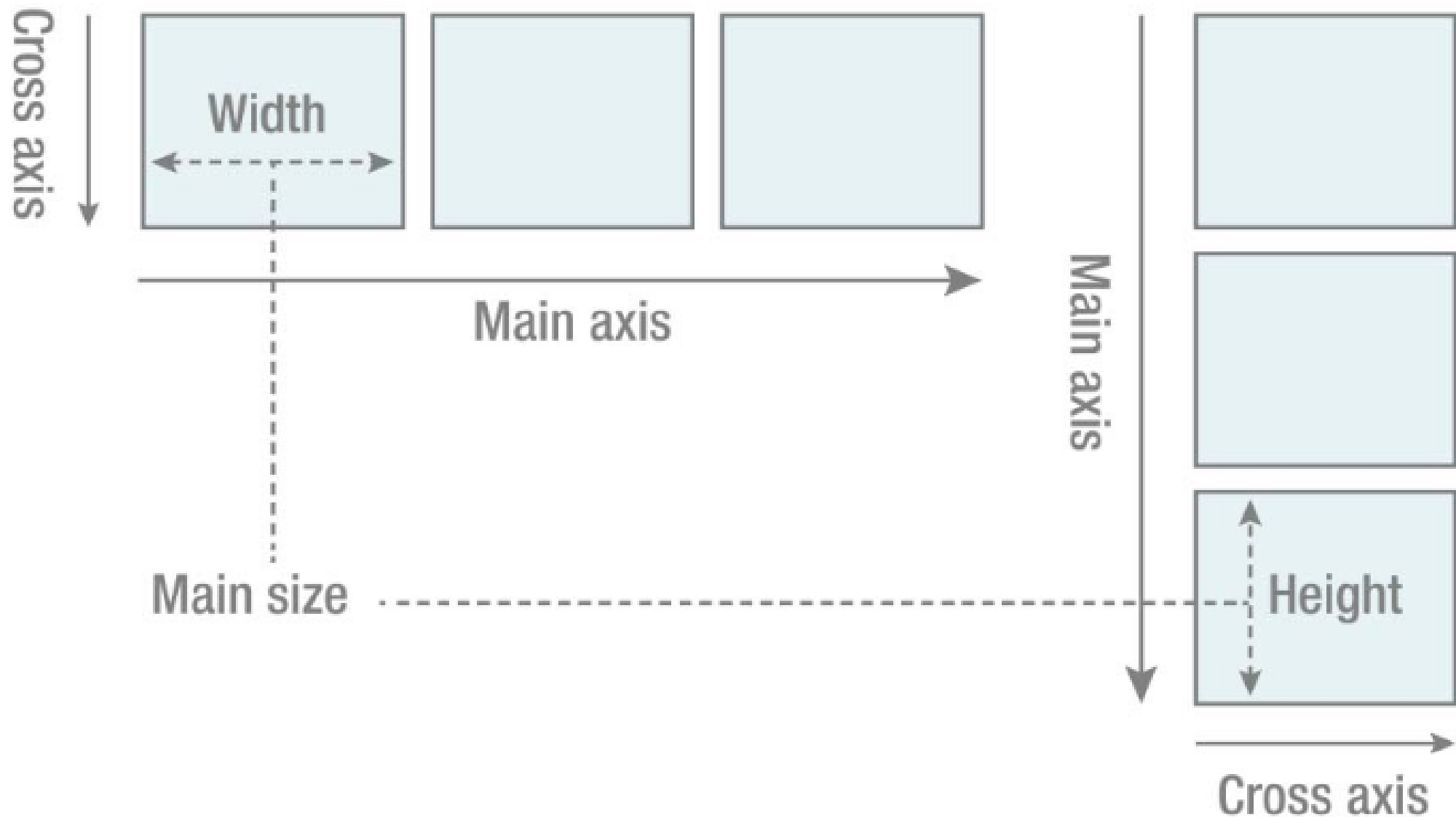
все **дочерние элементы** этого контейнера автоматически флекс элементы
элементы, вложенные уже в дочерние элементы **не будут являться флекс
элементами** по умолчанию

```
<section> <!-- флекс контейнер -->
  <div></div> <!-- дочерний флекс элемент -->
  <div></div> <!-- дочерний флекс элемент -->
  <div> <!-- дочерний флекс элемент -->
    <p>Первый абзац</p> <!-- не является флекс элементом -->
    <p>Второй абзац</p> <!-- не является флекс элементом -->
    <p>Третий абзац</p> <!-- не является флекс элементом -->
  </div>
</section>
```

МОДЕЛЬ FLEXBOX



РЕЖИМ СТРОКИ И КОЛОНКИ



Flex-контейнер не является блочным контейнером, поэтому для дочерних элементов не работают такие CSS-свойства, как **float**, **clear**, **vertical-align**. Также, на flex-контейнер не оказывают влияние свойства **column-***, создающие колонки в тексте и псевдоэлементы **::first-line** и **::first-letter**. Поля **margin** соседних flex-элементов не схлопываются. Процентные значения **margin** и **padding** вычисляются от внутреннего размера содержащего их блока



Порядок отображения flex-элементов и ориентация

Направление главной оси: **flex-direction**

row	Значение по умолчанию, слева направо (в rtl справа налево). Flex-элементы выкладываются в строку. Начало (main-start) и конец (main-end) направления главной оси соответствуют началу (inline-start) и концу (inline-end) оси строки (inline-axis).
row-reverse	Направление справа налево (в rtl слева направо). Flex-элементы выкладываются в строку относительно правого края контейнера (в rtl — левого).

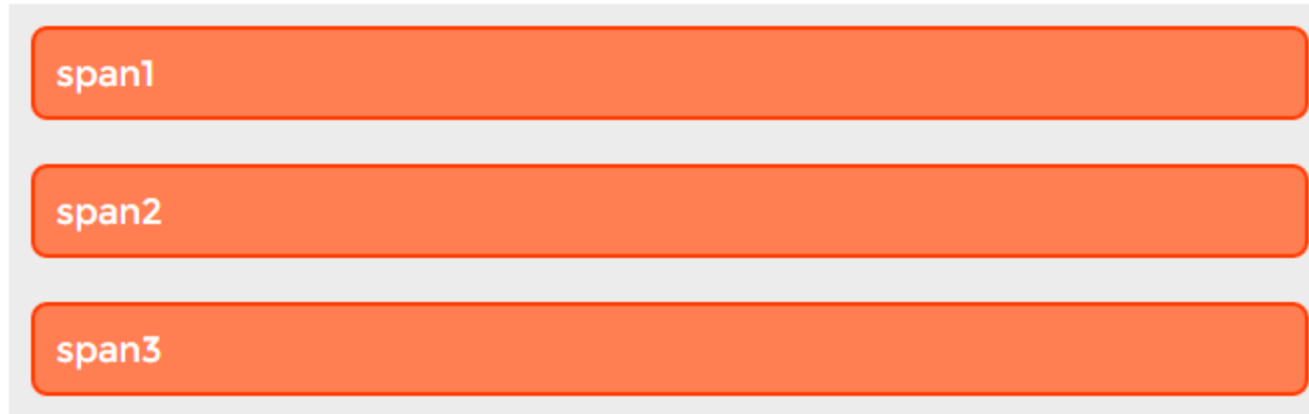
div3

div2

div1

column	Направление сверху вниз. Flex-элементы выкладываются в колонку.
column-reverse	Колонка с элементами в обратном порядке, снизу вверх.

```
flex-direction: column;
```



```
flex-direction: column-reverse;
```



Управление многострочностью flex-контейнера: **flex-wrap**

nowrap	Значение по умолчанию. Flex-элементы не переносятся, а располагаются в одну линию слева направо (в rtl справа налево).
wrap	Flex-элементы переносятся, располагаясь в несколько горизонтальных рядов (если не помещаются в один ряд) в направлении слева направо (в rtl справа налево).

`flex-wrap: wrap;`



wrap-reverse

Flex-элементы переносятся на новые линии, располагаясь в обратном порядке слева-направо, при этом перенос происходит снизу вверх.

```
flex-wrap: wrap-reverse;
```

span7

span8

span9

span10

span1

span2

span3

span4

span5

span6

Краткая запись направления и многострочности: **flex-flow**

Представляет собой сокращённую запись свойств flex-direction и flex-wrap.

направление	Указывает направление главной оси. Значение по умолчанию row.
многострочность	Задаёт многострочность поперечной оси. Значение по умолчанию nowrap.

```
.flex-container {  
  display: -webkit-flex;  
  -webkit-flex-flow: row wrap;  
  display: flex;  
  flex-flow: row wrap;  
}
```

Порядок отображения flex-элементов: **order**

Первоначально все flex-элементы имеют `order: 0`;

При указании значения от `-1` для элемента он перемещается в начало строки, значение `1` — в конец.

Если несколько flex-элементов имеют одинаковое значение `order`, они будут отображаться в соответствии с исходным порядком.



Гибкость flex-элементов

Flex-элемент будет полностью «негибок», если его значения **flex-grow** (свободное пространство между элементами) и **flex-shrink** (сжатие содержимого) равны нулю, и «гибкий» в противном случае.

Задание гибких размеров одним свойством: **flex**

Свойство является сокращённой записью свойств **flex-grow**, **flex-shrink** и **flex-basis**.

Значение по умолчанию: **flex: 0 1 auto;**. Можно указывать как одно, так и все три значения свойств.

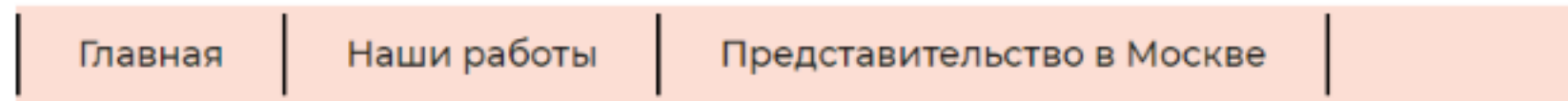
коэффициент растяжения	Коэффициент увеличения ширины flex-элемента относительно других flex-элементов.
коэффициент сужения	Коэффициент уменьшения ширины flex-элемента относительно других flex-элементов.
базовая ширина	Базовая ширина flex-элемента.
auto	Эквивалентно flex: 1 1 auto;.
none	Эквивалентно flex: 0 0 auto;.

Коэффициент роста: **flex-grow**

Свойство определяет коэффициент роста одного flex-элемента относительно других flex-элементов в flex-контейнере при распределении положительного свободного пространства.

Положительное целое или дробное число, устанавливающее коэффициент роста flex-элемента. Значение по умолчанию 0.

Если сумма значений flex-grow flex-элементов в строке меньше 1, они занимают менее 100% свободного пространства



```
li {flex-grow: 0;}
```



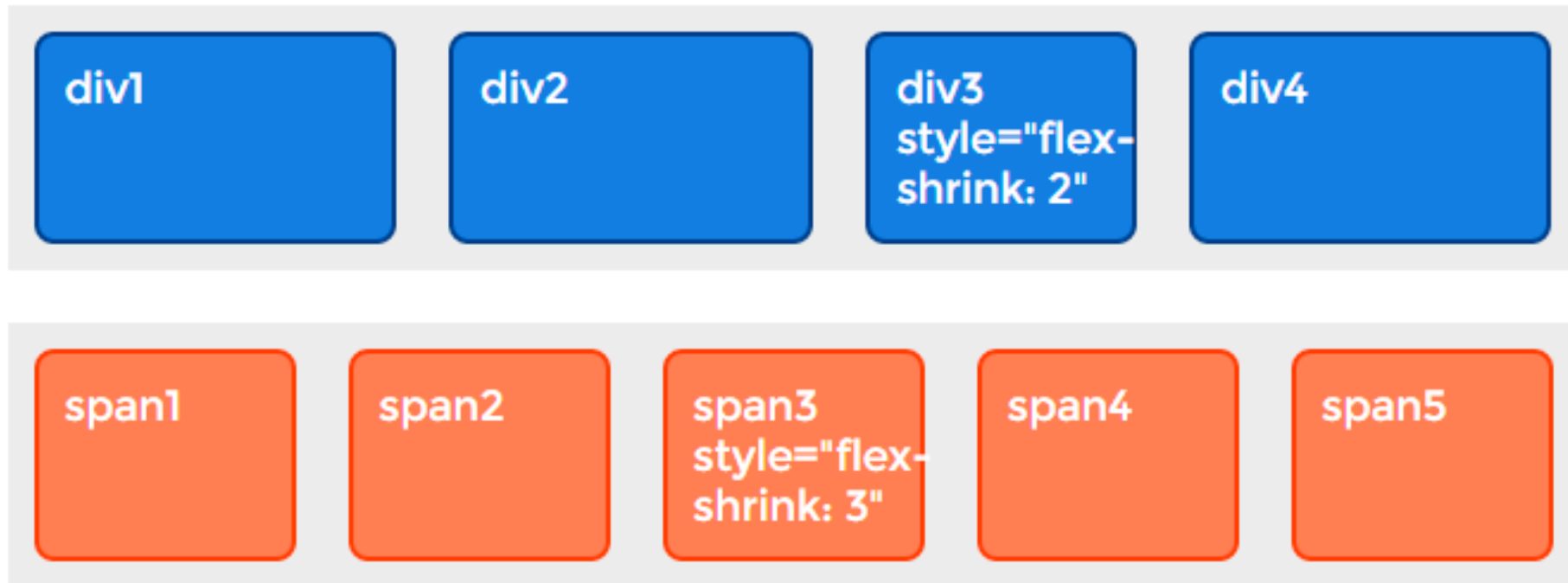
```
li {flex-grow: 1;}
```



```
li:last-child {flex-grow: 1;}
```

Коэффициент сжатия: **flex-shrink**

Свойство указывает коэффициент сжатия flex-элемента относительно других flex-элементов при распределении отрицательного свободного пространства. Умножается на базовый размер **flex-basis**. Отрицательное пространство распределяется пропорционально тому, насколько элемент может сжаться, поэтому, например, маленький flex-элемент не уменьшится до нуля, пока не будет заметно уменьшен flex-элемент большего размера.



Базовый размер: **flex-basis**

Свойство устанавливает начальный основной размер flex-элемента до распределения свободного пространства в соответствии с коэффициентами гибкости. Для всех значений, кроме auto и content, базовый гибкий размер определяется так же, как width в горизонтальных режимах записи. Процентные значения определяются относительно размера flex-контейнера, а если размер не задан, используемым значением для flex-basis являются размеры его содержимого.

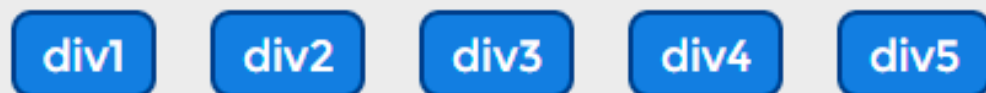
auto	Значение по умолчанию. Элемент получает базовый размер, соответствующий размеру его содержимого (если он не задан явно).
content	Определяет базовый размер в зависимости от содержимого flex-элемента.
длина	Базовый размер определяется так же, как для ширины и высоты. Задается в единицах длины.

Выравнивание по главной оси: **justify-content**

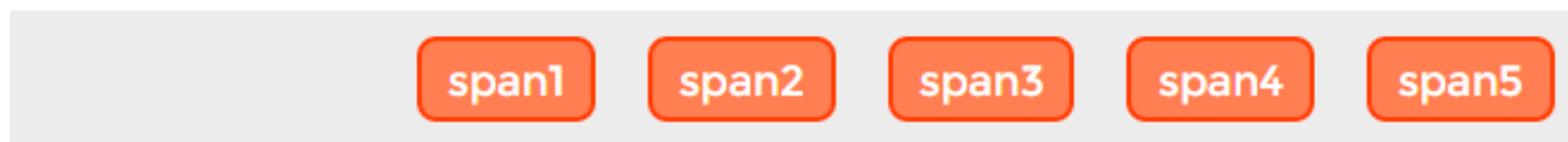
Свойство выравнивает flex-элементы по главной оси flex-контейнера, распределяя свободное пространство, незанятое flex-элементами. Когда элемент преобразуется в flex-контейнер, flex-элементы по умолчанию сгруппированы вместе (если для них не заданы поля margin). Промежутки добавляются после расчета значений margin и flex-grow. Если какие-либо элементы имеют ненулевое значение flex-grow или margin: auto;, свойство не будет оказывать влияния.

flex-start	Значение по умолчанию. Flex-элементы выкладываются в направлении, идущем от начальной линии flex-контейнера.
flex-end	Flex-элементы выкладываются в направлении, идущем от конечной линии flex-контейнера.
center	Flex-элементы выравниваются по центру flex-контейнера.
space-between	Flex-элементы равномерно распределяются по линии. Первый flex-элемент помещается вровень с краем начальной линии, последний flex-элемент — вровень с краем конечной линии, а остальные flex-элементы на линии распределяются так, чтобы расстояние между любыми двумя соседними элементами было одинаковым. Если оставшееся свободное пространство отрицательно или в строке присутствует только один flex-элемент, это значение идентично параметру flex-start.
space-around	Flex-элементы на линии распределяются так, чтобы расстояние между любыми двумя смежными flex-элементами было одинаковым, а расстояние между первым / последним flex-элементами и краями flex-контейнера составляло половину от расстояния между flex-элементами.

`justify-content: flex-start;`



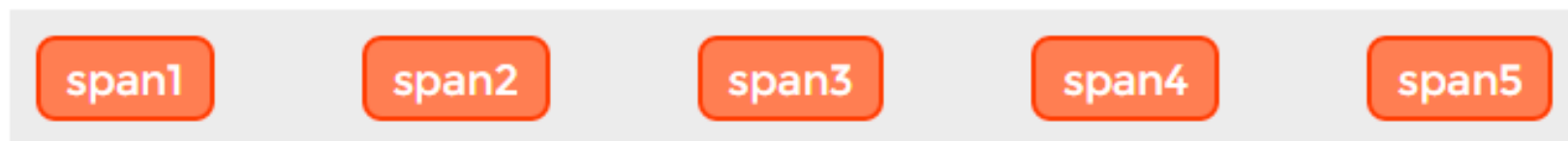
`justify-content: flex-end;`



`justify-content: center;`



`justify-content: space-between;`



`justify-content: space-around;`



Выравнивание по поперечной оси: **align-items** и **align-self**

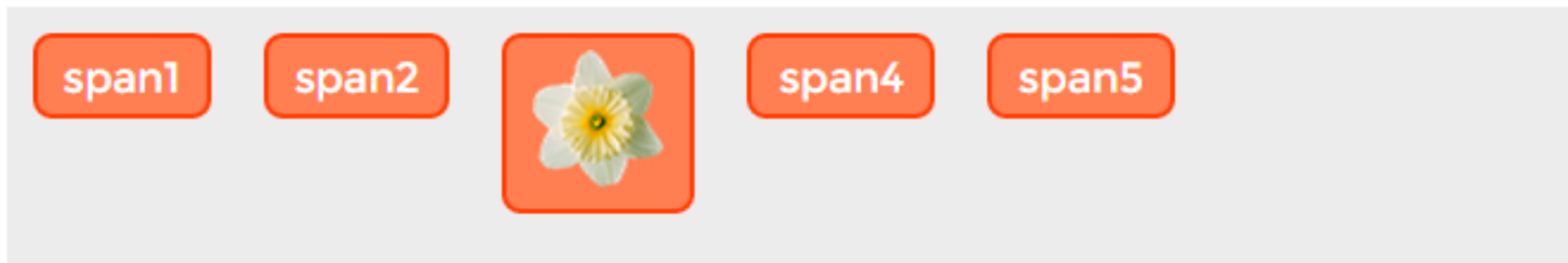
Flex-элементы можно выравнивать по поперечной оси текущей строки flex-контейнера. **align-items** устанавливает выравнивание для всех элементов flex-контейнера, включая анонимные flex-элементы.

align-self позволяет переопределить это выравнивание для отдельных flex-элементов. Если любое из поперечных margin flex-элемента имеет значение auto, align-self не имеет никакого влияния.

`align-items: stretch;`



`align-items: flex-start;`



`align-items: flex-end;`



```
align-items: center;
```



```
align-items: baseline;
```



Align-self

Свойство отвечает за выравнивание отдельно взятого flex-элемента по высоте flex-контейнера. Переопределяет выравнивание, заданное **align-items**.

`align-self: flex-start;`

div2

div3

div4

div5

`align-self: flex-end;`

span2

span3

span4

span5

`align-self: center;`

div2

div3

div4

div5

`align-self: baseline;`

span2

span3

span4

span5

`align-self: stretch;`

div2

div3

div4

div5

stretch

Если поперечный размер flex-элемента вычисляется как auto и ни одно из поперечных значений margin не равно auto, элемент растягивается. Значение по умолчанию.

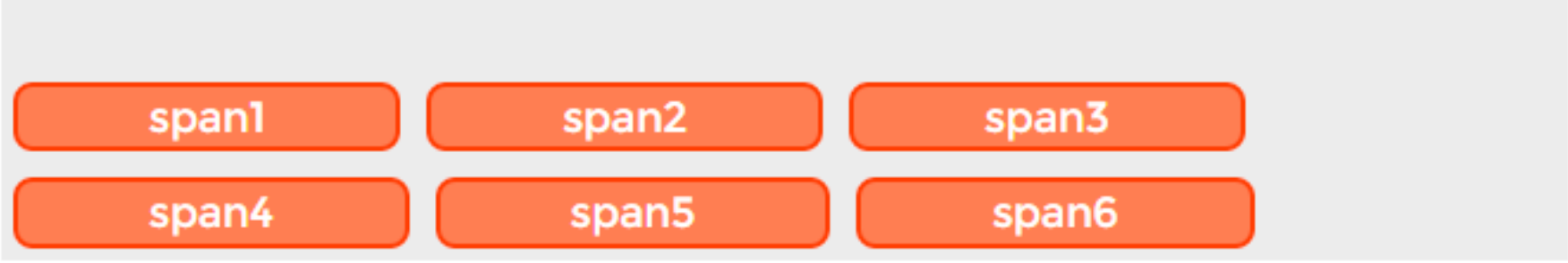
Выравнивание строк flex-контейнера: align-content

Свойство выравнивает строки в flex-контейнере при наличии дополнительного пространства на поперечной оси, аналогично выравниванию отдельных элементов на главной оси с помощью свойства justify-content. Свойство не влияет на однострочный flex-контейнер.

```
align-content: flex-start;
```

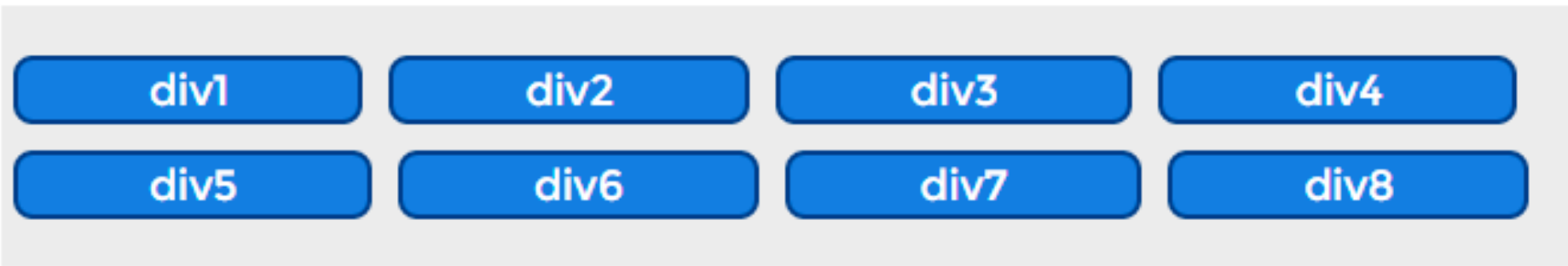


```
align-content: flex-end;
```



flex-start	Строки укладываются по направлению к началу flex-контейнера. Край первой строки помещается вплотную к краю flex-контейнера, каждая последующая — вплотную к предыдущей строке.
flex-end	Строки укладываются по направлению к концу flex-контейнера. Край последней строки помещается вплотную к краю flex-контейнера, каждая предыдущая — вплотную к последующей строке.

```
align-content: center;
```



```
align-content: space-between;
```



center	Строки укладываются по направлению к центру flex-контейнера. Строки расположены вплотную друг к другу и выровнены по центру flex-контейнера с равным расстоянием между начальным краем содержимого flex-контейнера и первой строкой и между конечным краем содержимого flex-контейнера и последней строкой.
space-between	Строки равномерно распределены в flex-контейнере. Если оставшееся свободное пространство отрицательно или в flex-контейнере имеется только одна flex-линия, это значение идентично flex-start. В противном случае край первой строки помещается вплотную к начальному краю содержимого flex-контейнера, край последней строки — вплотную к конечному краю содержимого flex-контейнера. Остальные строки распределены так, чтобы расстояние между любыми двумя соседними строками было одинаковым.

```
align-content: space-around;
```



```
align-content: stretch;
```



space-around	Строки равномерно распределены в flex-контейнере с половинным пробелом на обоих концах. Если оставшееся свободное пространство отрицательно, это значение идентично center. В противном случае строки распределяются таким образом, чтобы расстояние между любыми двумя соседними строками было одинаковым, а расстояние между первой / последней строками и краями содержимого flex-контейнера составляло половину от расстояния между строками.
stretch	Значение по умолчанию. Строки flex-элементов равномерно растягиваются, заполняя все доступное пространство. Если оставшееся свободное пространство отрицательно, это значение идентично flex-start. В противном случае свободное пространство будет разделено поровну между всеми строками, увеличивая их поперечный размер.